

Name : Mitali Mishra

Roll No : 154

Div : B

```
In [1]: import numpy as np  
import pandas as pd
```

```
In [2]: df=pd.read_csv('data_1_B.csv')
```

```
In [3]: df.head()
```

```
Out[3]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns



```
In [4]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     569 non-null    int64
1   diagnosis                             569 non-null    str
2   radius_mean                           569 non-null    float64
3   texture_mean                           569 non-null    float64
4   perimeter_mean                         569 non-null    float64
5   area_mean                             569 non-null    float64
6   smoothness_mean                       569 non-null    float64
7   compactness_mean                      569 non-null    float64
8   concavity_mean                        569 non-null    float64
9   concave points_mean                   569 non-null    float64
10  symmetry_mean                         569 non-null    float64
11  fractal_dimension_mean                 569 non-null    float64
12  radius_se                             569 non-null    float64
13  texture_se                             569 non-null    float64
14  perimeter_se                           569 non-null    float64
15  area_se                               569 non-null    float64
16  smoothness_se                         569 non-null    float64
17  compactness_se                        569 non-null    float64
18  concavity_se                          569 non-null    float64
19  concave points_se                     569 non-null    float64
20  symmetry_se                           569 non-null    float64
21  fractal_dimension_se                   569 non-null    float64
22  radius_worst                          569 non-null    float64
23  texture_worst                         569 non-null    float64
24  perimeter_worst                       569 non-null    float64
25  area_worst                            569 non-null    float64
26  smoothness_worst                      569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                       569 non-null    float64
29  concave points_worst                   569 non-null    float64
30  symmetry_worst                        569 non-null    float64
31  fractal_dimension_worst                569 non-null    float64
32  Unnamed: 32                           0 non-null      float64
dtypes: float64(31), int64(1), str(1)
memory usage: 147.4 KB

```

```
In [5]: df.isnull().sum()
```

```
Out[5]: id          0
        diagnosis   0
        radius_mean 0
        texture_mean 0
        perimeter_mean 0
        area_mean    0
        smoothness_mean 0
        compactness_mean 0
        concavity_mean 0
        concave points_mean 0
        symmetry_mean 0
        fractal_dimension_mean 0
        radius_se     0
        texture_se     0
        perimeter_se   0
        area_se        0
        smoothness_se  0
        compactness_se 0
        concavity_se   0
        concave points_se 0
        symmetry_se     0
        fractal_dimension_se 0
        radius_worst   0
        texture_worst  0
        perimeter_worst 0
        area_worst     0
        smoothness_worst 0
        compactness_worst 0
        concavity_worst 0
        concave points_worst 0
        symmetry_worst  0
        fractal_dimension_worst 0
        Unnamed: 32    569
        dtype: int64
```

```
In [6]: df.drop(columns=['id', 'Unnamed: 32'], inplace=True)
```

```
In [7]: df.describe()
```

Out[7]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	comp
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
std	3.524049	4.301036	24.298981	351.914129	0.014064	
min	6.981000	9.710000	43.790000	143.500000	0.052630	
25%	11.700000	16.170000	75.170000	420.300000	0.086370	
50%	13.370000	18.840000	86.240000	551.100000	0.095870	
75%	15.780000	21.800000	104.100000	782.700000	0.105300	
max	28.110000	39.280000	188.500000	2501.000000	0.163400	

8 rows × 30 columns

In [8]: `from sklearn.preprocessing import LabelEncoder`
In [9]: `le = LabelEncoder()
df['diagnosis'] = le.fit_transform(df['diagnosis'])`
In [10]: `df.head()`

Out[10]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	1	17.99	10.38	122.80	1001.0	0.11840
1	1	20.57	17.77	132.90	1326.0	0.08474
2	1	19.69	21.25	130.00	1203.0	0.10960
3	1	11.42	20.38	77.58	386.1	0.14250
4	1	20.29	14.34	135.10	1297.0	0.10030

5 rows × 31 columns


In [11]: `X = df.drop('diagnosis', axis=1)
y = df['diagnosis']`
In [12]: `from sklearn.model_selection import train_test_split`In [13]: `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state`

```
In [14]: from sklearn.preprocessing import StandardScaler
```

```
In [15]: sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
In [16]: from tensorflow.keras.models import Sequential
         from tensorflow.keras.layers import Dense
```

```
In [17]: classifier = Sequential()
```

```
In [18]: classifier.add(Dense(units=6, activation='relu', input_dim=X_train.shape[1]))
```

c:\Users\admin\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [19]: classifier.add(Dense(units=6, activation='relu'))
```

```
In [20]: classifier.add(Dense(units=1, activation='sigmoid'))
```

```
In [21]: classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```
In [22]: classifier.fit(X_train, y_train, batch_size=23, epochs=56)
```

Epoch 1/56
20/20  **4s** 7ms/step - accuracy: 0.4220 - loss: 0.6313

Epoch 2/56
20/20  **0s** 6ms/step - accuracy: 0.5275 - loss: 0.5777

Epoch 3/56
20/20  **0s** 5ms/step - accuracy: 0.6571 - loss: 0.5414

Epoch 4/56
20/20  **0s** 6ms/step - accuracy: 0.7670 - loss: 0.5126

Epoch 5/56
20/20  **0s** 8ms/step - accuracy: 0.8484 - loss: 0.4865

Epoch 6/56
20/20  **0s** 13ms/step - accuracy: 0.8945 - loss: 0.4613

Epoch 7/56
20/20  **1s** 15ms/step - accuracy: 0.9165 - loss: 0.4340

Epoch 8/56
20/20  **0s** 14ms/step - accuracy: 0.9385 - loss: 0.4016

Epoch 9/56
20/20  **0s** 6ms/step - accuracy: 0.9429 - loss: 0.3643

Epoch 10/56
20/20  **0s** 10ms/step - accuracy: 0.9538 - loss: 0.3186

Epoch 11/56
20/20  **0s** 12ms/step - accuracy: 0.9538 - loss: 0.2698

Epoch 12/56
20/20  **0s** 13ms/step - accuracy: 0.9538 - loss: 0.2238

Epoch 13/56
20/20  **0s** 6ms/step - accuracy: 0.9582 - loss: 0.1848

Epoch 14/56
20/20  **0s** 12ms/step - accuracy: 0.9626 - loss: 0.1554

Epoch 15/56
20/20  **0s** 14ms/step - accuracy: 0.9648 - loss: 0.1342

Epoch 16/56
20/20  **0s** 7ms/step - accuracy: 0.9692 - loss: 0.1190

Epoch 17/56
20/20  **0s** 15ms/step - accuracy: 0.9714 - loss: 0.1077

Epoch 18/56
20/20  **0s** 6ms/step - accuracy: 0.9714 - loss: 0.0990

Epoch 19/56
20/20  **0s** 9ms/step - accuracy: 0.9736 - loss: 0.0923

Epoch 20/56
20/20  **0s** 13ms/step - accuracy: 0.9736 - loss: 0.0869

Epoch 21/56
20/20  **0s** 6ms/step - accuracy: 0.9736 - loss: 0.0824

Epoch 22/56
20/20  **0s** 14ms/step - accuracy: 0.9758 - loss: 0.0787

Epoch 23/56
20/20  **0s** 6ms/step - accuracy: 0.9780 - loss: 0.0757

Epoch 24/56
20/20  **0s** 7ms/step - accuracy: 0.9780 - loss: 0.0728

Epoch 25/56
20/20  **0s** 6ms/step - accuracy: 0.9802 - loss: 0.0703

Epoch 26/56
20/20  **0s** 13ms/step - accuracy: 0.9802 - loss: 0.0682

Epoch 27/56
20/20  **0s** 11ms/step - accuracy: 0.9802 - loss: 0.0663

Epoch 28/56
20/20  **0s** 7ms/step - accuracy: 0.9802 - loss: 0.0644

Epoch 29/56
20/20  **0s** 11ms/step - accuracy: 0.9824 - loss: 0.0628

Epoch 30/56
20/20  **0s** 7ms/step - accuracy: 0.9824 - loss: 0.0611

Epoch 31/56
20/20  **0s** 11ms/step - accuracy: 0.9824 - loss: 0.0598

Epoch 32/56
20/20  **0s** 14ms/step - accuracy: 0.9868 - loss: 0.0583

Epoch 33/56
20/20  **0s** 11ms/step - accuracy: 0.9868 - loss: 0.0571

Epoch 34/56
20/20  **0s** 12ms/step - accuracy: 0.9868 - loss: 0.0559

Epoch 35/56
20/20  **0s** 7ms/step - accuracy: 0.9868 - loss: 0.0548

Epoch 36/56
20/20  **0s** 10ms/step - accuracy: 0.9868 - loss: 0.0536

Epoch 37/56
20/20  **0s** 7ms/step - accuracy: 0.9868 - loss: 0.0522

Epoch 38/56
20/20  **0s** 6ms/step - accuracy: 0.9868 - loss: 0.0512

Epoch 39/56
20/20  **0s** 7ms/step - accuracy: 0.9868 - loss: 0.0500

Epoch 40/56
20/20  **0s** 6ms/step - accuracy: 0.9868 - loss: 0.0491

Epoch 41/56
20/20  **0s** 12ms/step - accuracy: 0.9868 - loss: 0.0484

Epoch 42/56
20/20  **0s** 7ms/step - accuracy: 0.9868 - loss: 0.0473

Epoch 43/56
20/20  **0s** 7ms/step - accuracy: 0.9868 - loss: 0.0466

Epoch 44/56
20/20  **0s** 10ms/step - accuracy: 0.9868 - loss: 0.0456

Epoch 45/56
20/20  **0s** 8ms/step - accuracy: 0.9868 - loss: 0.0447

Epoch 46/56
20/20  **0s** 9ms/step - accuracy: 0.9890 - loss: 0.0438

Epoch 47/56
20/20  **0s** 6ms/step - accuracy: 0.9890 - loss: 0.0431

Epoch 48/56
20/20  **0s** 7ms/step - accuracy: 0.9890 - loss: 0.0423

Epoch 49/56
20/20  **0s** 8ms/step - accuracy: 0.9890 - loss: 0.0416

Epoch 50/56
20/20  **0s** 6ms/step - accuracy: 0.9890 - loss: 0.0409

Epoch 51/56
20/20  **0s** 6ms/step - accuracy: 0.9912 - loss: 0.0400

Epoch 52/56
20/20  **0s** 9ms/step - accuracy: 0.9934 - loss: 0.0392

Epoch 53/56
20/20  **0s** 8ms/step - accuracy: 0.9934 - loss: 0.0385

Epoch 54/56
20/20  **0s** 8ms/step - accuracy: 0.9934 - loss: 0.0380

Epoch 55/56
20/20  **0s** 7ms/step - accuracy: 0.9934 - loss: 0.0371

Epoch 56/56
20/20  **0s** 9ms/step - accuracy: 0.9934 - loss: 0.0367

Out[22]: <keras.src.callbacks.history.History at 0x1bf5c316270>

In [23]: `y_pred = classifier.predict(X_test)`

4/4  0s 81ms/step

In [24]: `y_pred = (y_pred > 0.5)`

In [25]: `from sklearn.metrics import confusion_matrix`

`cm = confusion_matrix(y_test, y_pred)`

`print(cm)`

```
[[70  1]
 [ 2 41]]
```

In [26]: `from sklearn.metrics import accuracy_score`

In [27]: `accuracy = accuracy_score(y_test, y_pred)`

`print("Accuracy:", accuracy)`

Accuracy: 0.9736842105263158